CSC317 Computer Graphics

Term Test 1 · Solutions walkthrough

Raster images · Ray–cone intersection · Ray tracing

Term Test 1 at a glance

6
Q1 · Raster images

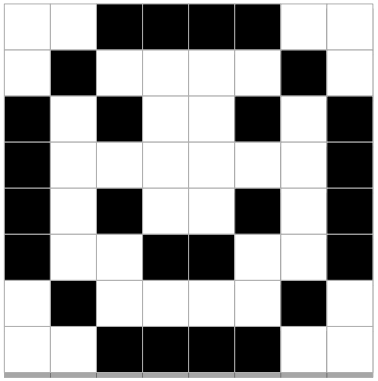
11
Q2 · Ray–cone intersection

8
Q3 · Ray tracing

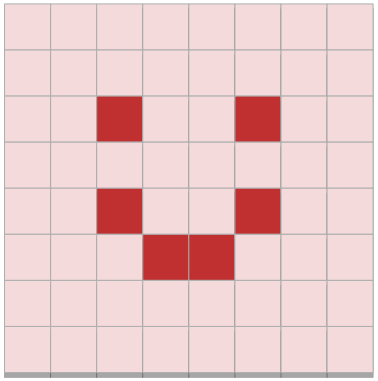
25 marks

Q1 · Comparing two small images

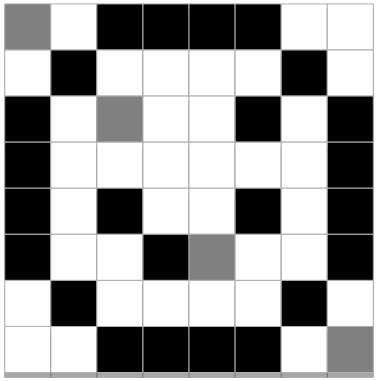
E: gray level *l*



E: importance a



I: gray level I



E is an 8×8 emoticon. Each pixel has a gray level I and an importance a, both in [0, 1]

I is another 8×8 image, with just gray levels

At least one pixel of E has a = 1

Q1(a) · The question [2 marks]

Write a difference function f(E, I) between 0 and 1

f = 0 when E and I are identical

f = 1 when every pixel is as different as possible, **weighted by importance**

How far apart are E and I?

Q1(a) · One pixel at a time

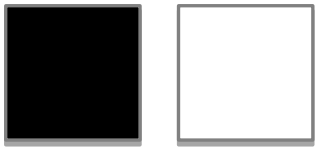
Difference of one pixel: $|E.I - I.I|$

- same gray: 0
- black vs white: 1

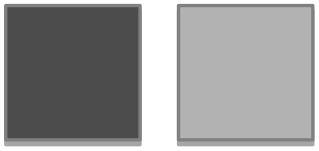
Multiply by importance a, so important pixels count more



$|0 - 0| = 0$



$|0 - 1| = 1$



$|0.3 - 0.7| = 0.4$

Q1(a) · Add up, then divide

$$f = \frac{\text{sum over pixels of } a \cdot |E.l - I.l|}{\text{sum over pixels of } a}$$

Identical: the top is 0, so f = 0

Maximally different: every |…| = 1, top = bottom, so f = 1

The bottom is never 0, because at least one a = 1

Grading · Q1(a)

GRADING

Full marks: 0 for identical, 1 for maximally different, weighted by a

No alpha in the formula: −1

Assumed an alpha for I: −0.5

Missing or wrong normalization: −0.5

- dividing by 64 only reaches 1 when every a = 1

Only defining the two extreme cases: 0

Q1(b) · The question [2 marks]

A simpler function compares only averages:

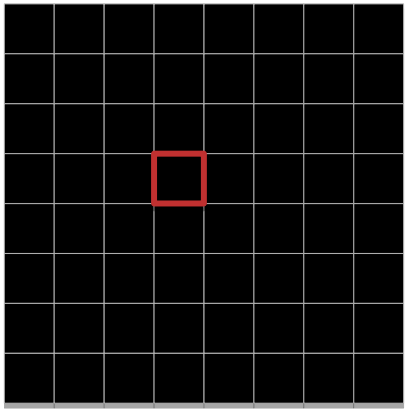
- g(E, I) = |E_avg − I_avg|

Find images where f and g disagree completely:

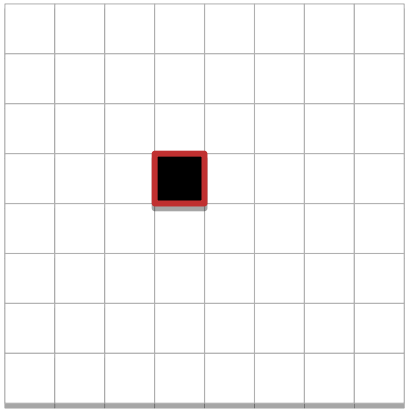
- (i) f = 0 but g = 63/64
- (ii) f = 1 but g = 0

Q1(b)(i) · f = 0 but g = 63/64

E: all black



I: white, except that pixel



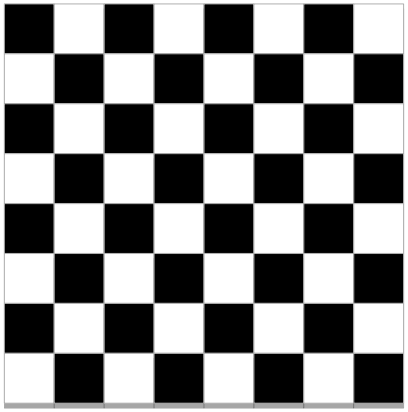
Only the red pixel has a = 1. All others have a = 0

f: the only pixel that counts matches, so f = 0

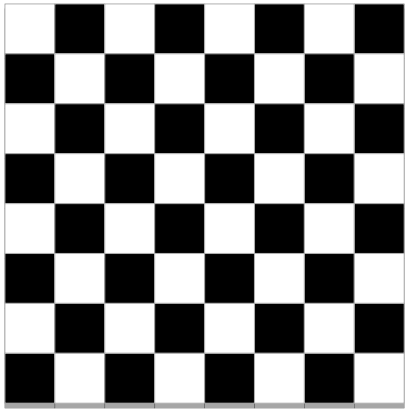
g: E_avg = 0, I_avg = 63/64, so g = 63/64

Q1(b)(ii) · f = 1 but g = 0

E: checkerboard



I: the negative



Every pixel of E has a = 1

f: every pixel differs by 1, so f = 1

g: both are half black, half white. Same average, so g = 0

Grading · Q1(b)

GRADING

1 mark for each part: a clear construction

Clear means it cannot be read as a wrong E or I

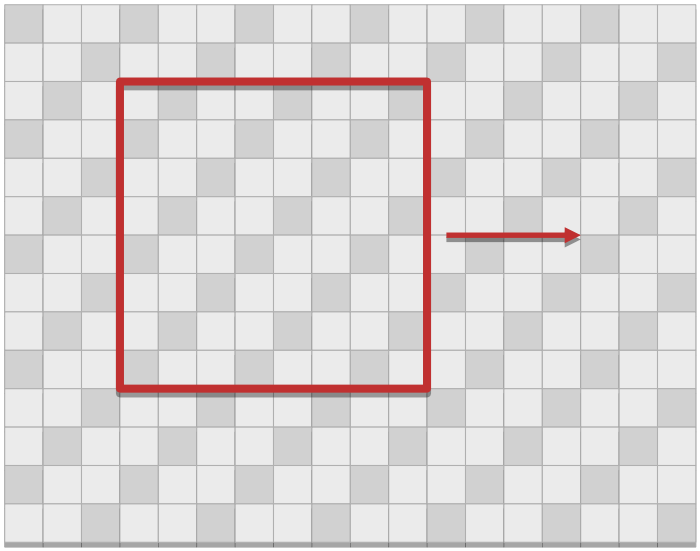
If (a) was wrong, (b) was graded with your own f from (a)

Q1(c) · The question [2 marks]

A big m×n image J

Find **every** 8×8 region that matches E

Match means f(E, region) < e



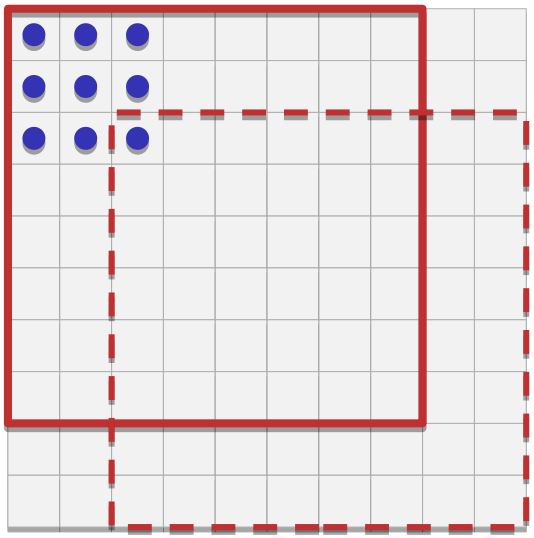
slide the 8×8 window over J

Q1(c) · Slide a window

Try every top-left corner where 8×8 still fits

A 10×10 image has 3×3 = 9 windows

An 8×8 image has exactly **1**



blue dots: the 9 possible top-left corners

Q1(c) · Solution

```
matches = []
for i in 0 .. m-8:          # inclusive
    for j in 0 .. n-8:
        W = J[i:i+8, j:j+8]
        if f(E, W) < e:
            matches.append((i, j))
return matches
```

m = 8: i runs from 0 to 0, exactly once ✓

Grading · Q1(c)

GRADING

1 mark: loop over every 8×8 window

1 mark: check f(E, window) < e

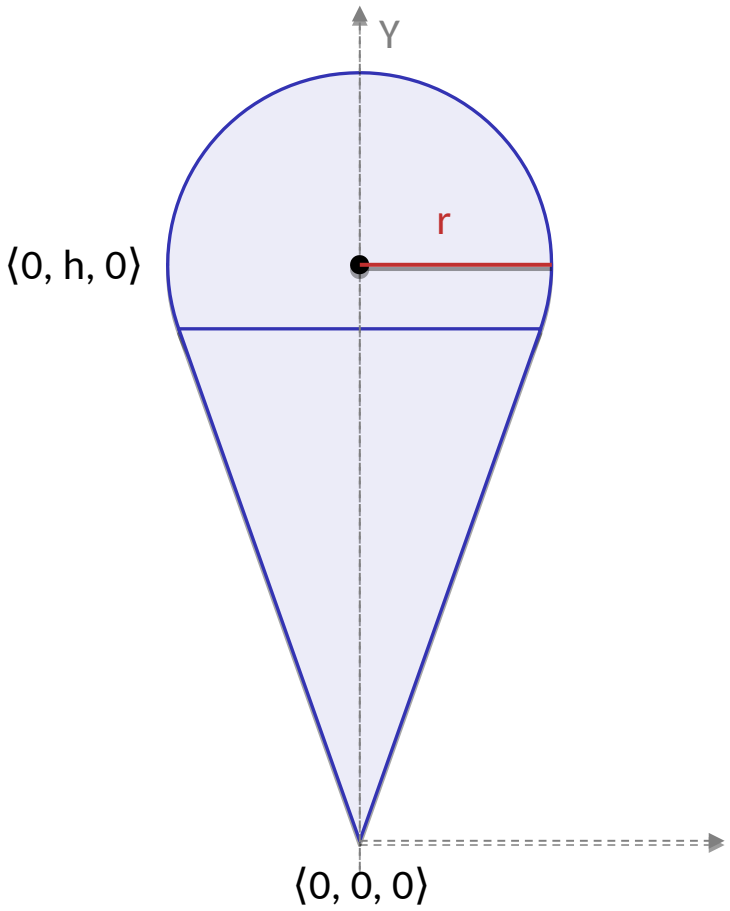
No explicit sliding window: no marks

Off-by-one boundaries: 0.5 out of 1

Only one match returned, or nothing returned: −0.5

Returning a count was accepted

Q2 · The ice-cream shape [11 marks]



Cone: tip at the origin, opening up along Y

Sphere: radius r, centre ⟨0, h, 0⟩

The sphere just touches the cone side

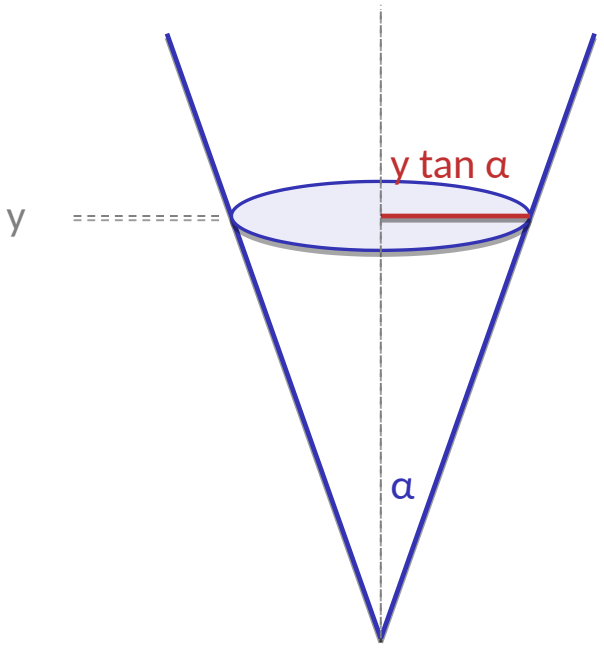
Q2 · Where the cone equation comes from

$$x^2 + z^2 - y^2 \tan^2\alpha = 0$$

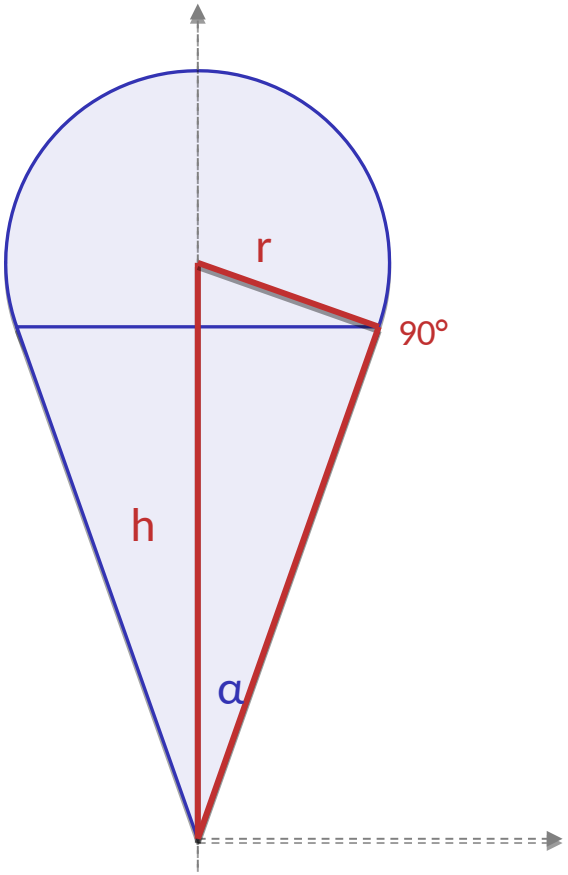
Cut the cone at height y: you get a circle

Its radius is $y \cdot \tan \alpha$

A point is on that circle when $x^2 + z^2 = (y \tan \alpha)^2$



Q2(a) · The half-angle α [1 mark]



The radius to the touching point is at 90° to the cone side

Right triangle: hypotenuse h, side opposite α is r

$$\sin \alpha = r / h$$

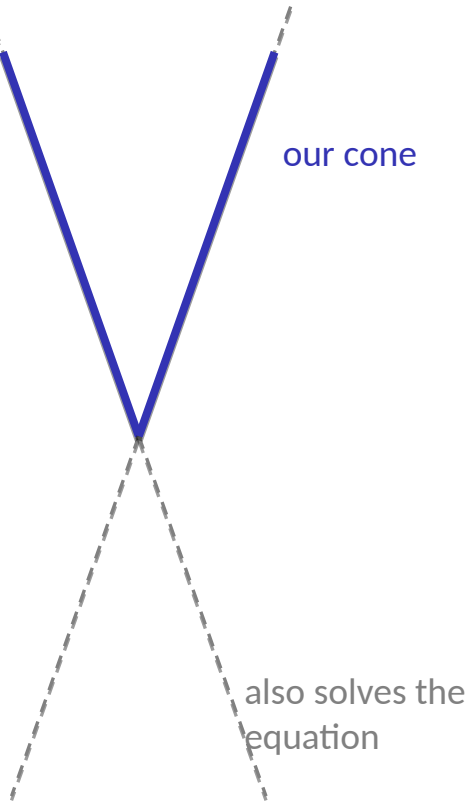
$$\alpha = \arcsin(r / h)$$

Q2(b) · The equation is too big [1 mark]

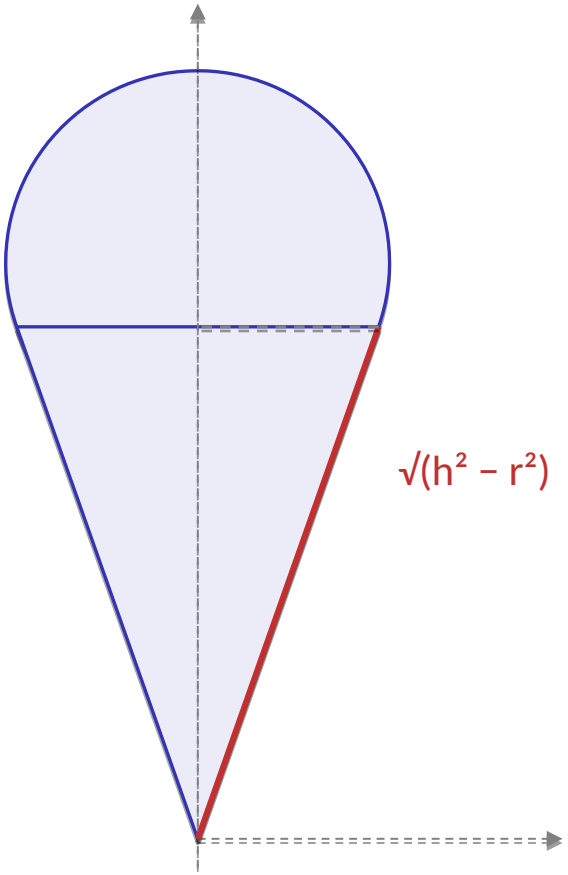
The equation describes an **infinite double cone**

Our cone is only the solid blue part

So we must also check the height



Q2(b) · How high does the cone go?



Cone side up to the touching point: $\sqrt{(h^2 - r^2)}$

Its height: $\sqrt{(h^2 - r^2)} \cdot \cos \alpha$

$\cos \alpha = h / \sqrt{(h^2 - r^2)}$

Together we get

$$0 \le y \le h - r^2/h$$

equivalently $h - r \sin \alpha$ *or* $h \cos^2 \alpha$

Grading · Q2(a)–(b)

GRADING

(a) $\alpha = \arcsin(r/h)$. $\sin\alpha = r/h$ also accepted

(b) Full marks for the correct top height, in any form

- e.g. $P.y < h - r\sin\alpha$, $\|P\| < \sqrt{(h^2 - r^2)}$, $P.y < h\cos^2\alpha$

(b) $P.y < h$ only: 0.5

- the cone stops where it meets the sphere, not at h

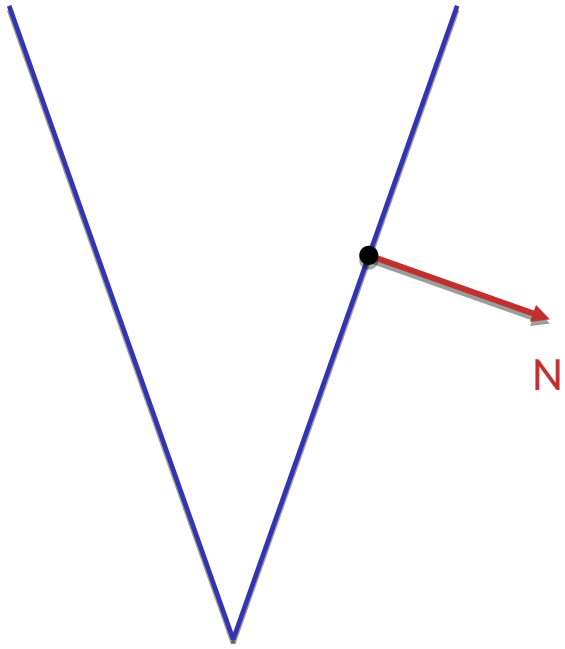
Q2(c) · The surface normal [1 mark]

For a surface F(P) = 0, the normal is the gradient of F

The gradient points where F grows fastest: straight out of the surface

$$F = x^2 + z^2 - y^2 \tan^2\alpha$$
$$\nabla F = (2x, -2y \tan^2\alpha, 2z)$$
$$N = (P.x, -P.y \tan^2\alpha, P.z)$$

No need to normalize



Grading · Q2(c)

GRADING

Full marks: the gradient of the cone equation

- e.g. $N = \langle P.x, -P.y \tan^2\alpha, P.z \rangle$

Arithmetic error, or not derived: 0.5

- e.g. only "take the gradient", or half-finished

Unnormalized N was accepted

Q2(d) · The question [4 marks]

```
def coneIntersect(S, D, α, r, h, t) -> int:
```

We are given

- A ray: start S, direction D
- The cone: α, r, h
- An empty list t

We must

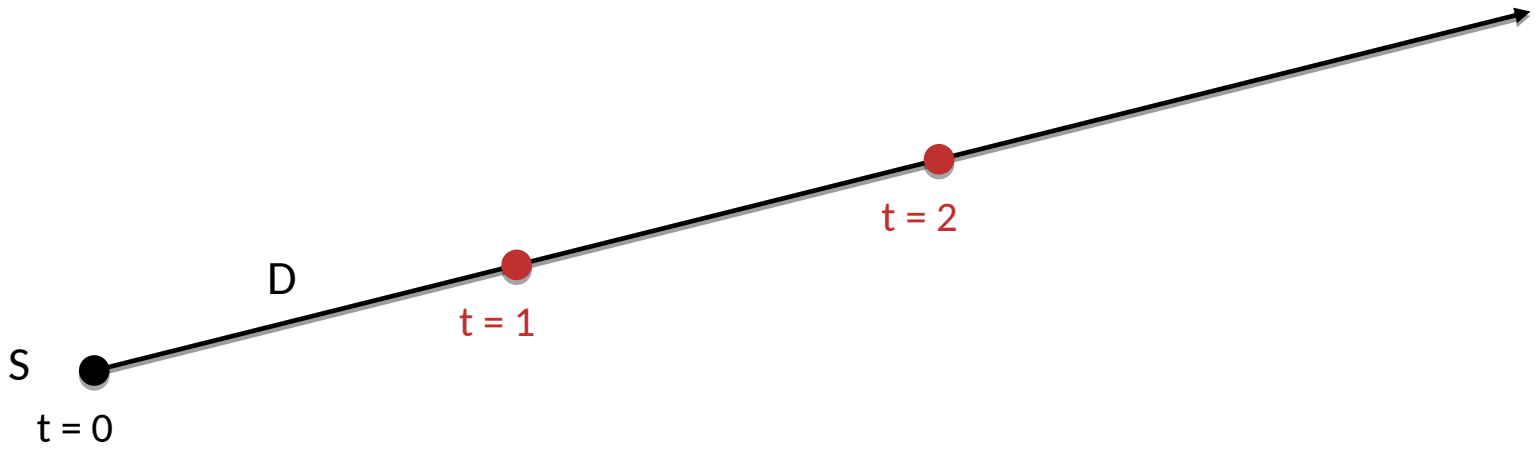
- Find where the ray hits our cone
- Put each hit into t, nearest first
- Return the **number** of hits

Q2(d) · Step 1: put the ray into the equation

Every point on the ray is P(t) = S + t D

A hit is a t where P(t) satisfies the cone equation

$$(S.x + t\ D.x)^2 + (S.z + t\ D.z)^2 - \tan^2\alpha\ (S.y + t\ D.y)^2 = 0$$



Q2(d) · Step 2: it is a quadratic in t

```
A = D.x² + D.z² − tan²α · D.y²
B = 2 (S.x D.x + S.z D.z − tan²α · S.y D.y)
C = S.x² + S.z² − tan²α · S.y²

A t² + B t + C = 0
```

$B^2 - 4AC < 0$: no real roots, the ray misses

Otherwise: up to 2 roots, up to 2 hits

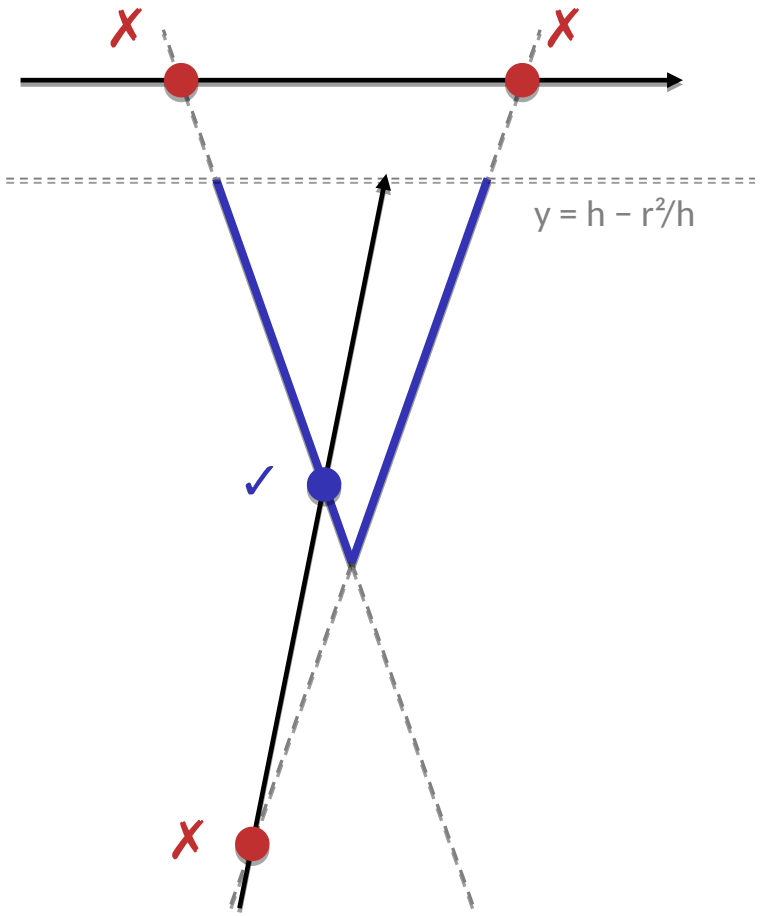
Q2(d) · Step 3: keep only hits on our cone

For each root t, the hit height is

- $y = S.y + t \cdot D.y$

Keep it only if $0 \le y \le h - r^2/h$

Check the **hit point's** y, not t



Q2(d) · Solution

```
def coneIntersect(S, D, α, r, h, t) -> int:
    k = tan(α) ** 2
    A = D.x**2 + D.z**2 - k*D.y**2
    B = 2*(S.x*D.x + S.z*D.z - k*S.y*D.y)
    C = S.x**2 + S.z**2 - k*S.y**2
    disc = B*B - 4*A*C
    if disc < 0: return 0
    count = 0
    for tr in the real roots:
        y = S.y + tr*D.y      # hit height
        if tr >= 0 and 0 <= y <= h - r*r/h:
            t.append(tr)
            count += 1
    t.sort()
    return count
```

Grading · Q2(d)

GRADING

1	2	1
plug S + tD into the cone	height check on the hit point	return the count

Bonus +1: checking the discriminant, or exact roots

List t: +0.5 for filling it, +0.5 for sorting it

- the bonuses cannot go above full marks

Grading · Q2(d) common slips

GRADING

Height check on t instead of the hit point's y: −0.5

Only P.y < h: 1.5 out of 2 for the height check

Wrong, but consistent with your (b): 1 out of 2

Count computed but not returned: −0.5

Height check done in (e) instead: still counted

Q2(e) · The whole ice cream [3 marks]

Hits on the ice cream = hits on the sphere + hits on the cone

```
def iceCreamIntersect(S, D, α, r, h, t) -> int:
    sphereIntersect(S, D, r, h, t)     # adds sphere hits
    coneIntersect(S, D, α, r, h, t)    # adds cone hits
    t.sort()
    return len(t)
```

Both calls add to the **same** list t

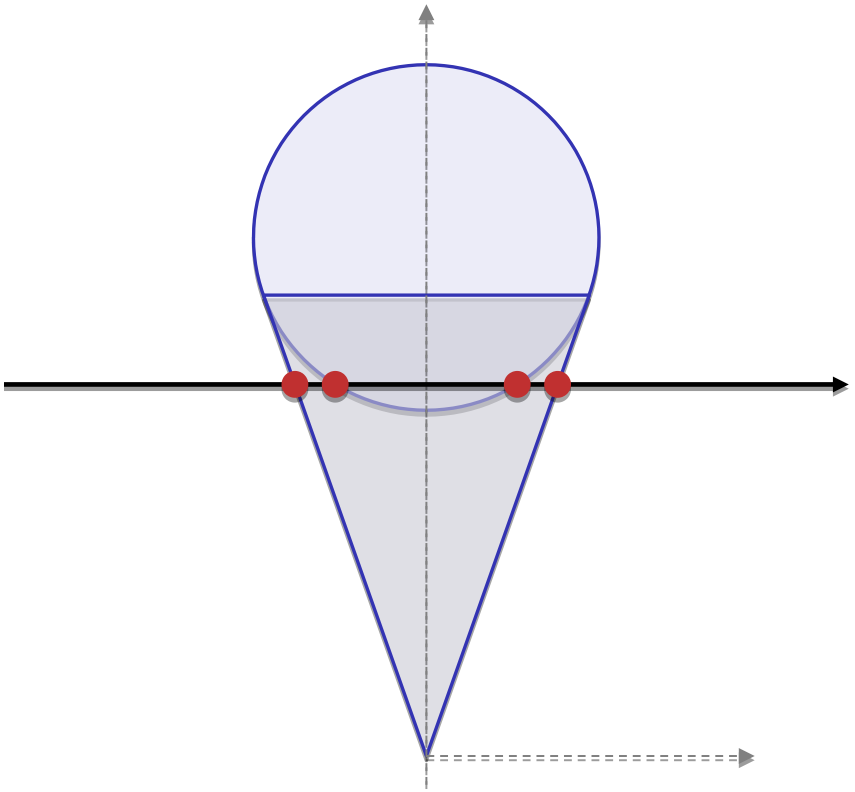
So coneIntersect must add to t, never reset it

Q2(f) · At most how many hits? [1 mark]

4

2 on the cone + 2 on the sphere

The ray passes just below where they meet



Grading · Q2(e)–(f)

(e) 1 mark: correct sphereIntersect and coneIntersect calls

(e) 2 marks: return the correct count. Not returned: −1

(e) coneIntersect must add to t, not reset it

(f) 1 mark: 4 hits, with a correct drawing

(f) several drawings, one correct: 0.5

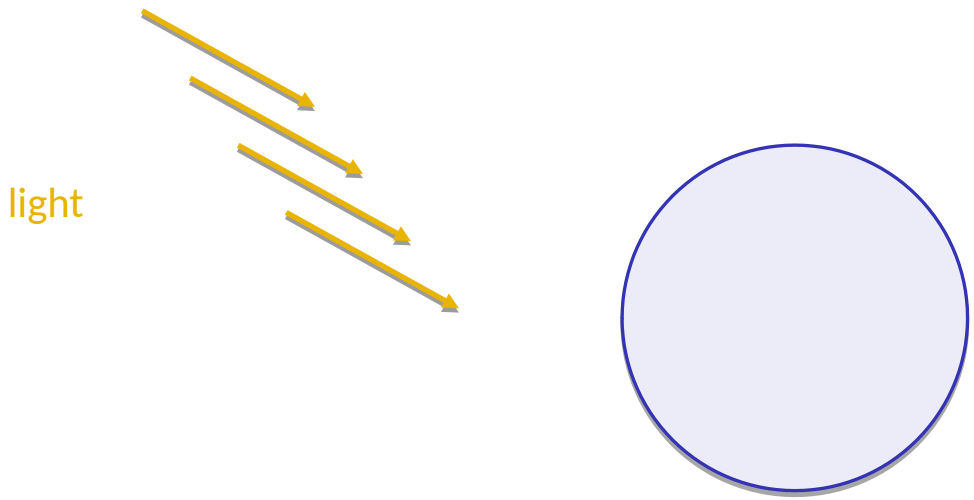
Q3(a) · A perfect mirror ball [2 marks]

A directional light shines on a perfect mirror sphere

Take a point P facing the light

Show: infinitely many eye positions see P as a highlight

Q3 says: NO marks without proper reasoning



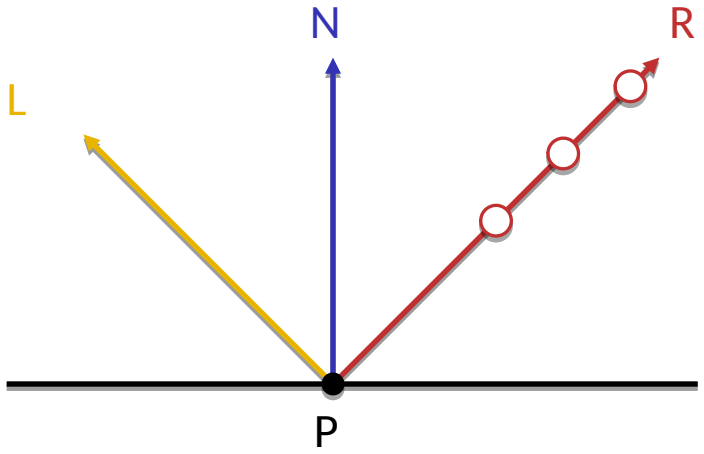
Q3(a) · Solution

A perfect mirror sends light out in **one** direction R

- $R = 2(N \cdot L)N - L$

Any eye on the ray $P + sR$ ($s > 0$) sees the highlight

A ray has infinitely many points



every eye on R sees the highlight

Grading · Q3(a)

GRADING

1 mark: a perfect mirror reflects in exactly one direction

1 mark: infinitely many eyes along R from P

Fixing L, e.g. L = V = R: 1 mark

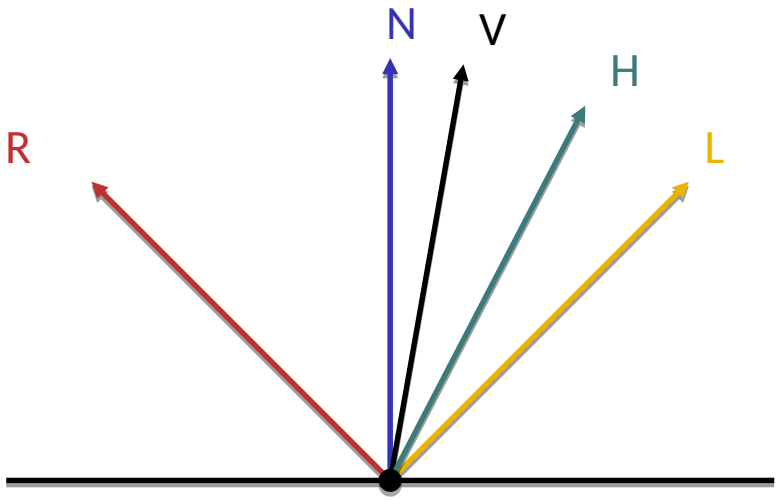
Lambert's law is for diffuse surfaces. Not this question

Q3(b) · Phong vs Blinn [2 marks]

Both make shiny highlights:

- Phong uses V·R: how close the eye is to the mirror direction
- Blinn uses N·H: H is halfway between L and V

Build vectors where V·R and N·H are clearly different



Q3(b) · Solution: use numbers

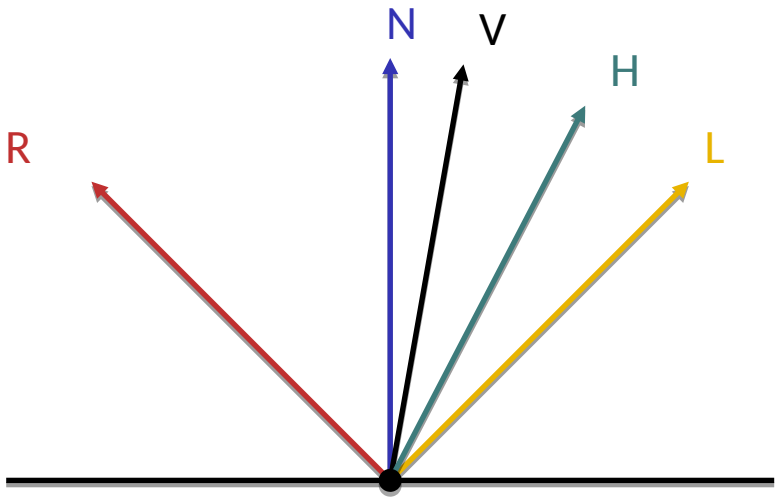
N straight up. L at 45°, V at 10°, same side

R is at −45°:

- V·R = cos 55° ≈ **0.57**

H is at 27.5°:

- N·H = cos 27.5° ≈ **0.89**



All unit vectors, all above the surface

Grading · Q3(b)

GRADING

2 marks: a valid configuration, drawn or with numbers

Degenerate vectors (below the surface, zero): 1

Only relative angles, degenerate cases not ruled out: 1

V and L on opposite sides, angles unspecified: 0

V, L, N should be unit vectors

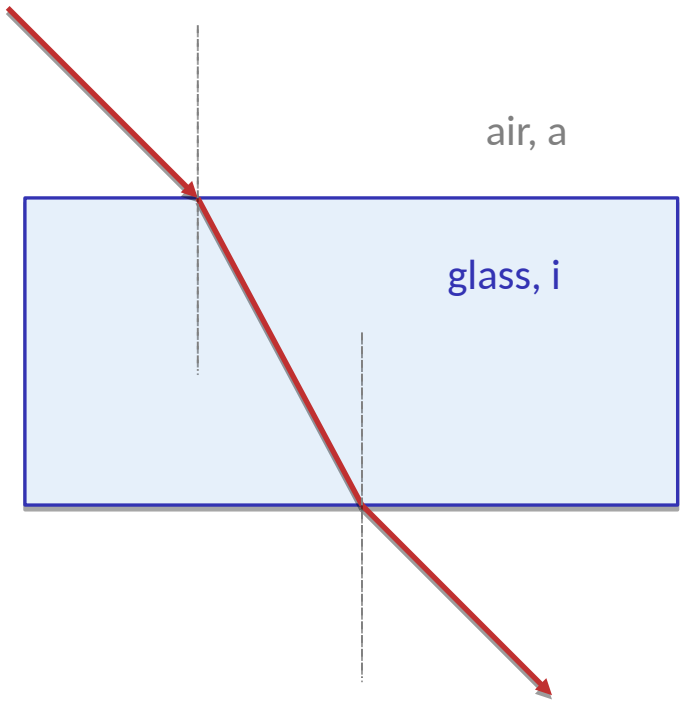
Q3(c) · Light through glass [2 marks]

Snell's law: $a \sin \theta_a = i \sin \theta_i$

- a: air, i: glass, θ: angle to the normal

Entering the glass, the ray bends toward the normal. Is i > a?

When is the exit ray parallel to the entry ray?



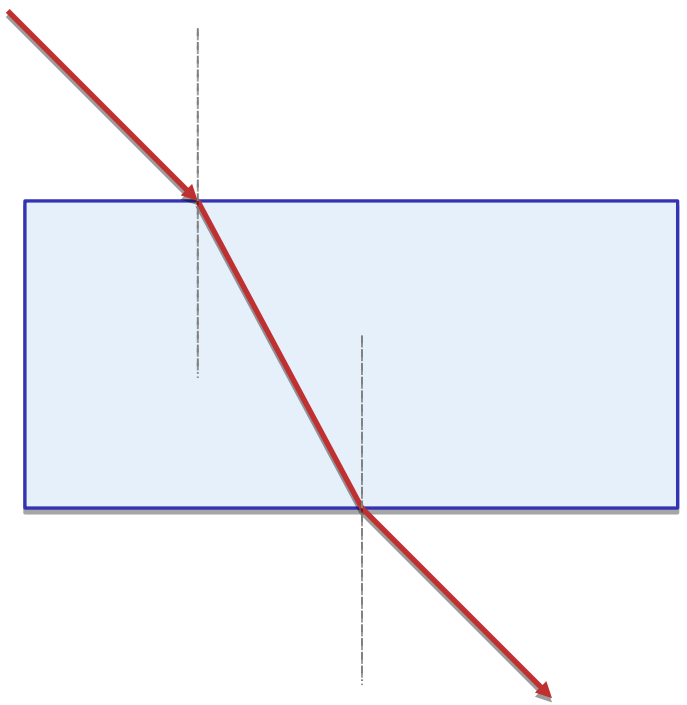
Q3(c) · Solution

Toward the normal: $\theta_i < \theta_a$

Smaller angle, smaller sine: $\sin\theta_i < \sin\theta_a$

Snell: $a \sin\theta_a = i \sin\theta_i$, so **$i > a$**

Parallel faces: the exit undoes the entry, so exit $\parallel$ entry



parallel faces: in and out rays are parallel

Grading · Q3(c)

1 mark: i > a, with reasoning from Snell's law

1 mark: say the entry and exit faces are parallel

"i = a" as a parallel condition was also credited

"It bends toward the normal" alone: not enough

- that only restates the question

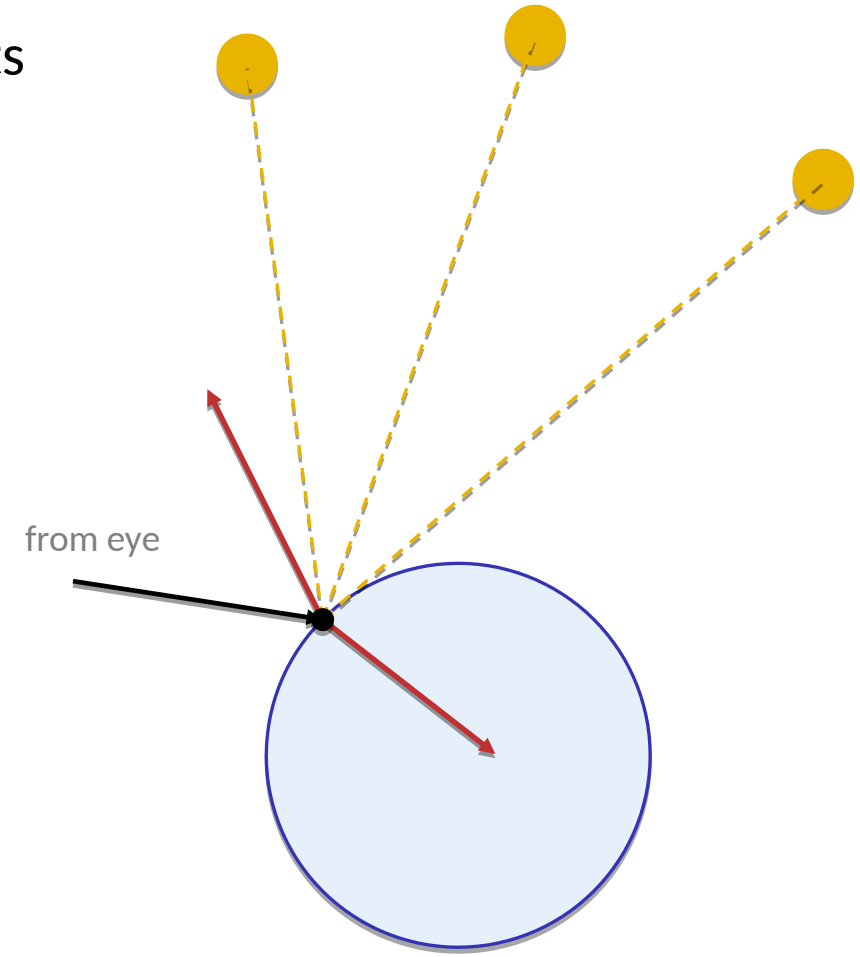
Q3(d) · How many new rays? [2 marks]

3 lights, transparent shiny objects

1 reflection ray

1 refraction ray

3 shadow rays, one per light



= 5 rays

Grading · Q3(d)

GRADING

1 mark: 2 rays from reflection and refraction

1 mark: 3 shadow rays

Each count needs its cause, e.g. "3 shadow rays, one per light"

Right pieces, wrong total (e.g. 6): 0.5